

BALANCEAMENTO DE ÁRVORES: AVL, RUBRO-NEGRAS E B-TREES

Amanda Bialer Ingham 24.01621-7

Disciplina: ECM306 – Estruturas de Dados

Instituto Mauá de Tecnologia

Data: Junho de 2026

Resumo

As árvores binárias de busca são estruturas de dados amplamente utilizadas na computação para representar conjuntos de dados de forma hierárquica, permitindo operações eficientes de inserção, remoção e busca. Contudo, sua eficiência depende do balanceamento da estrutura: uma árvore degenerada pode degenerar em desempenho equivalente a uma lista encadeada, com complexidade $O(n)$. O presente artigo apresenta os principais métodos para se obter árvores balanceadas, descrevendo os conceitos, propriedades e mecanismos de rebalanceamento das Árvores AVL, das Árvores Rubro-Negras (Red-Black Trees) e das Árvores B-Tree. Cada abordagem é discutida em termos de suas invariantes, operações de rotação e ajuste de cores, além de suas aplicações práticas. Conclui-se que a escolha entre os métodos depende do contexto de uso, dos padrões de acesso e das restrições de memória do sistema.

Palavras-chave: Árvores balanceadas; Árvore AVL; Árvore Rubro-Negra; B-Tree; Estruturas de dados.

Abstract

Binary search trees are widely used data structures in computer science for representing data sets hierarchically, enabling efficient insertion, deletion, and search operations. However, their efficiency critically depends on the balance of the structure: a degenerate tree may perform as poorly as a linked list, with $O(n)$ complexity. This paper presents the main methods for obtaining balanced trees, describing the concepts, properties, and rebalancing mechanisms of

AVL Trees, Red-Black Trees, and B-Trees. Each approach is discussed in terms of its invariants, rotation and color-adjustment operations, and practical applications in real systems. It is concluded that the choice among methods depends on the use context, access patterns, and memory constraints of the system.

Keywords: Balanced trees; AVL tree; Red-Black tree; B-Tree; Data structures.

1. Introdução

As árvores binárias de busca (ABB) constituem uma das mais fundamentais estruturas de dados na computação. Sua organização hierárquica permite que operações de busca, inserção e remoção sejam realizadas em tempo médio $O(\log n)$, desde que a árvore permaneça aproximadamente balanceada. Quando isso não ocorre, a altura da árvore pode crescer linearmente com o número de elementos, degradando todas as operações para $O(n)$ (CORMEN et al., 2022).

O problema do desbalanceamento ocorre de quando os dados são inseridos em sequência crescente ou decrescente, caso em que a ABB assume a forma de uma lista encadeada. Para diminuir esse problema, foram propostas diversas estruturas de árvores auto-balanceadas ao longo das últimas décadas, cada uma com suas garantias de desempenho.

Este artigo tem como objetivo apresentar e comparar os três métodos mais relevantes para o balanceamento de árvores: as Árvores AVL, as Árvores Rubro-Negras e as Árvores B-Tree. A compreensão dessas estruturas é essencial para o desenvolvimento de sistemas eficientes, desde bancos de dados relacionais e compiladores até sistemas operacionais e motores de busca.

2. Fundamentos do Balanceamento de Árvores

Uma árvore binária de busca é dita balanceada quando a diferença entre as alturas de suas subárvores esquerda e direita é limitada por alguma constante. Uma ABB é classificada como perfeitamente balanceada quando todos os seus nós folha estão no mesmo nível ou no máximo em dois níveis consecutivos. Na prática, a construção de tais árvores é muito custosa após cada operação de modificação, razão pela qual estruturas com critérios de balanceamento menos rígidos são adotadas (SEEDGEWICK; WAYNE, 2011).

As principais invariantes utilizadas nas estruturas estudadas neste artigo são: (i) o critério de altura, utilizado pelas Árvores AVL; (ii) o critério cromático, utilizado pelas Árvores Rubro-Negras; e (iii) o critério de ocupação de nós, utilizado pelas Árvores B-Tree. Em todos os casos, o objetivo é garantir que a altura total da árvore seja $O(\log n)$, assegurando desempenho mais eficiente nas operações.

3. Árvores AVL

3.1 Definição e Propriedades

Proposta por Adelson-Velsky e Landis em 1962, a Árvore AVL foi a primeira estrutura de dados auto-balanceada da literatura. Sua propriedade fundamental é que, para todo nó da árvore, o fator de balanceamento pertence ao conjunto $\{-1, 0, +1\}$. Essa restrição garante que a altura máxima de uma AVL com n nós seja aproximadamente $1,44 \cdot \log_2(n)$, assegurando desempenho $O(\log n)$ para busca, inserção e remoção (KNUTH, 1998).

3.2 Mecanismos de Rebalanceamento

Após cada inserção ou remoção, o fator de balanceamento de cada nó ancestral é recalculado no caminho de volta à raiz. Caso algum nó apresente fator de balanceamento fora do intervalo permitido, são realizadas rotações para restaurar a propriedade AVL.

Existem quatro casos de desbalanceamento, denominados: Rotação Simples à Direita (caso LL), Rotação Simples à Esquerda (caso RR), Rotação Dupla Esquerda-Direita (caso LR) e Rotação Dupla Direita-Esquerda (caso RL). As rotações simples envolvem a troca de posição entre um nó e seu filho; as rotações duplas realizam duas rotações consecutivas para corrigir situações de desbalanceamento em zigue-zague.

3.3 Complexidade e Aplicações

As operações de busca, inserção e remoção em Árvores AVL têm complexidade de $O(\log n)$ tanto no melhor quanto no pior caso. Por manterem um balanceamento controlado, as AVL são eficientes em cenários de leitura intensiva. São amplamente utilizadas em implementações de dicionários e em alguns bancos de dados relacionais.

4. Árvores Rubro-Negras

4.1 Definição e Propriedades

As Árvores Rubro-Negras foram introduzidas por Rudolf Bayer em 1972 sob o nome de árvores B simétricas, e posteriormente reformuladas por Guibas e Sedgwick em 1978 na forma conhecida atualmente.

Uma Árvore Rubro-Negra é uma árvore em que cada nó possui um atributo de cor (vermelho ou preto) e satisfaz as seguintes propriedades invariantes: (1) Todo nó é vermelho ou preto; (2) A raiz é preta; (3) Todo nó folha é preto; (4) Se um nó é vermelho, então ambos os seus filhos são pretos (não existem dois nós vermelhos consecutivos no mesmo caminho); (5) Para cada nó, todos os caminhos simples desse nó até as folhas descendentes contêm o mesmo número de nós pretos propriedade da altura preta (CORMEN et al., 2022).

4.2 Operações de Inserção e Remoção

Na inserção, o novo nó é colorido de vermelho. Caso as propriedades sejam violadas, dois tipos de correção são aplicados: recoloração (quando o tio do nó inserido é vermelho) e rotações (quando o tio é preto). Ao todo, são necessárias no máximo $O(\log n)$ recolorações e no máximo duas rotações por inserção.

A remoção é mais complexa e pode exigir múltiplas correções para restaurar a altura-negra. No pior caso, são realizadas $O(\log n)$ recolorações e três rotações. A altura de uma Árvore Rubro-Negra com n nós é no máximo $2 \cdot \log_2(n + 1)$, garantindo desempenho $O(\log n)$ em todas as operações.

4.3 Aplicações

Devido ao seu equilíbrio entre eficiência nas operações de leitura e escrita, as Árvores Rubro-Negras são utilizadas para o gerenciamento de processos e memória virtual, e em diversas bibliotecas de linguagens de programação modernas.

5. Árvores B-Tree

5.1 Definição e Propriedades

As Árvores B (B-Trees) são estruturas de dados generalizadas que estendem a ABB binária para permitir que cada nó contenha múltiplas chaves e múltiplos filhos. Propostas por Bayer e McCreight em 1972, foram especialmente concebidas para sistemas em que o acesso a dados em disco é o principal gargalo de desempenho (BAYER; McCREIGHT, 1972).

Uma B-Tree de grau mínimo t satisfaz as seguintes propriedades: (i) todo nó não-raiz possui no mínimo $t - 1$ chaves; (ii) todo nó possui no máximo $2t - 1$ chaves; (iii) todos os nós folha estão no mesmo nível; (iv) um nó com k chaves possui exatamente $k + 1$ filhos (para nós internos).

5.2 Operações e Rebalanceamento

Na inserção, quando um nó está cheio (possui $2t - 1$ chaves), ele é dividido em dois nós antes da descida recursiva, processo denominado split. Na remoção, se o nó alvo possui menos do que t chaves após a operação, ocorre uma operação de fusão (merge) ou redistribuição de chaves com um nó irmão. Todas as operações têm complexidade $O(\log n)$, e o número de acessos ao disco é minimizado pela alta ordem dos nós.

5.3 Variantes e Aplicações

A variante mais conhecida é a Árvore B+ (B+ Tree), na qual apenas os nós folha armazenam os dados efetivos, enquanto os nós internos armazenam apenas chaves de roteamento. Os nós folha são encadeados em uma lista duplamente ligada, o que facilita varreduras sequenciais eficientes.

As B-Trees e suas variantes são amplamente utilizadas em sistemas gerenciadores de bancos de dados relacionais (como PostgreSQL, MySQL e Oracle) e sistemas de arquivos (NTFS, ext4, HFS+), em razão da sua eficiência em cenários de acesso a disco e grande volume de dados.

6. Comparação entre os Métodos

A Tabela 1 sintetiza as principais características dos três métodos estudados.

Tabela 1 – Comparação entre métodos de balanceamento

Característica	AVL	Rubro-Negra	B-Tree
Complexidade (pior caso)	$O(\log n)$	$O(\log n)$	$O(\log_m n)$
Altura máxima	$1,44 \log_2 n$	$2 \log_2(n+1)$	$\log_m n$
Rotações por inserção	≤ 2	≤ 2	—
Rotações por remoção	$O(\log n)$	≤ 3	—
Nível de balanceamento	Estrito	Moderado	Perfeito
Overhead de memória	2 bits/nó	1 bit/nó	Alto (múltiplas chaves)

Característica	AVL	Rubro-Negra	B-Tree
Uso principal	Memória principal	Memória principal	Armazenamento em disco
Exemplos de uso	Tabelas de símbolos	STL C++, Java TreeMap	SGBDs (PostgreSQL, MySQL)

A principal diferença entre as Árvores AVL e Rubro-Negras está no rigor do balanceamento: AVL mantém uma altura estritamente menor, o que favorece buscas, porém impõe mais rotações nas inserções e remoções. As Rubro-Negras oferecem melhor desempenho amortizado em cargas de trabalho mistas (SEEDGEWICK; WAYNE, 2011).

As B-Trees se diferenciam pelo suporte a múltiplas chaves por nó, o que as torna ideais para sistemas de armazenamento secundário, onde o custo por acesso a disco é muito superior ao custo de processamento em memória RAM.

7. Conclusão

O balanceamento de árvores é um problema central em ciência da computação, com impacto direto na eficiência de sistemas que dependem de operações rápidas de busca, inserção e remoção. Este artigo apresentou três das principais abordagens para este problema: as Árvores AVL, as Árvores Rubro-Negras e as Árvores B-Tree.

Verificou-se que as Árvores AVL oferecem o balanceamento mais rígido, sendo preferíveis em sistemas com alta frequência de consultas. As Árvores Rubro-Negras alcançam um equilíbrio superior entre inserções, remoções e buscas, sendo amplamente adotadas em sistemas operacionais e bibliotecas de linguagens de programação. Já as Árvores B-Tree, especialmente em sua variante B+, se destacam como a solução canônica para indexação em bancos de dados e sistemas de arquivos.

A escolha da estrutura adequada deve considerar o perfil da carga de trabalho, a localidade de referência dos dados e as restrições de memória do ambiente. O estudo comparativo dessas estruturas evidencia que não existe uma solução universalmente superior: cada método possui vantagens específicas que o tornam mais adequado para determinadas aplicações.

Como perspectiva de trabalhos futuros, destacam-se o estudo das Árvores Treap, das Skip Lists e das estruturas de acesso por cache, que representam direções relevantes na pesquisa atual de estruturas de dados.

Referências Bibliográficas

- ADELSON-VELSKY, G. M.; LANDIS, E. M. An algorithm for the organization of information. *Soviet Mathematics Doklady*, v. 3, p. 1259–1263, 1962.
- BAYER, R.; McCREIGHT, E. M. Organization and maintenance of large ordered indexes. *Acta Informatica*, v. 1, n. 3, p. 173–189, 1972.
- CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 4. ed. Cambridge: MIT Press, 2022.
- GUIBAS, L. J.; SEDGEWICK, R. A dichromatic framework for balanced trees. In: *Proceedings of the 19th Annual Symposium on Foundations of Computer Science*, 1978. p. 8–21.
- KNUTH, D. E. *The Art of Computer Programming: Sorting and Searching*. v. 3. 2. ed. Boston: Addison-Wesley, 1998.
- SEDEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Boston: Addison-Wesley, 2011.
- ZIVIANI, N. *Projeto de Algoritmos: com Implementações em Java e C++*. 3. ed. São Paulo: Cengage Learning, 2010.